

Confronto di robustezza delle strategie di localizzazione

App: angular-spotify. Valori espressi come *mutazioni effettuate / mutazioni totali*.

Obsolescence e **Not Compiled** dipendono dalla mutazione (non dal locatore): sono quindi **identiche per tutte le strategie** nella stessa colonna.

1. Mutazioni generate tramite analisi statica

Tecnica della tesi precedente (operatori meccanici). 266 mutazioni.

Locator	Success	Fragility	Obsolescence	Not Compiled
Relative	117/266	23/266	68/266	58/266
Absolute	105/266	35/266	68/266	58/266
Robula	128/266	12/266	68/266	58/266
Katalon	95/198 *	2/198 *	68/266	58/266
Selenium	133/266	7/266	68/266	58/266
Robula+	131/266	9/266	68/266	58/266

* Katalon ha Success/Fragility su **198**: escluso da un test case (68 mutazioni) per un problema di formato pre-esistente del suo locatore (prefisso xpath=), non legato alle mutazioni. Obsolescence e Not-Compiled dipendono dalla mutazione (non dal locatore): restano quelle del set (68/266, 58/266) anche per Katalon.

2. Mutazioni generate da LLM secondo il modello statico (parametrico)

L'LLM applica gli 11 tipi di mutazione del modello statico (a-k). 256 mutazioni testate.

Locator	Success	Fragility	Obsolescence	Not Compiled
Relative	173/256	14/256	15/256	54/256
Absolute	172/256	15/256	15/256	54/256
Robula	183/256	4/256	15/256	54/256
Katalon	183/256	4/256	15/256	54/256
Selenium	177/256	10/256	15/256	54/256
Robula+	177/256	10/256	15/256	54/256

Serie ampliata da 161 a 256 mutazioni per equiparare il numero a quello dello static (266): il confronto è ora bilanciato (256 vs 266). La classifica di fragilità resta invariata (Absolute > Relative > Selenium/Robula+ > Robula/Katalon).

Prompt usato (parametrico) — system

```
# Prompt – Parametric Angular Mutation (LLM strategy, type-driven)

## Role
You are a front-end developer working on an Angular 19 template. In this task you apply ONE
specific KIND of change (a "mutation type") to the template. The exact mutation type to apply
is given in the user message, with its definition and a generic before/after example.

You are told WHICH type of change to make, but NOT which element: you decide autonomously
WHERE in the template to apply it, choosing a different, plausible location in each variant.

## Task
Produce N variants of the template provided in the user message. In EACH variant:
- apply exactly one instance of the requested mutation type, to ONE element of your choosing;
- pick a different target element / location across the N variants (do not repeat the same edit);
- the rest of the template stays identical to the original.

## Application independence (CRITICAL)
Operate ONLY on the markup you are given. Do NOT assume anything about which application this is,
its domain, features or data. Use exclusively the tags, classes and attributes already present in
the provided HTML (when the mutation type requires a new name/value, derive a plausible generic
one – never an application-specific term). The same approach must work on any Angular template.

## Validity constraints (correctness rules)
1. The result MUST still be valid Angular 19 and compile.
2. Do NOT alter the syntax of control flow or bindings: `@if`, `@for`, `*ngxLet`, `[prop]`,
  `(event)`, `[(two-way)]`, and `{ { }}` interpolations must stay valid.
3. Do NOT touch attributes starting with `x-test` (they are test ground truth).
4. Apply ONLY the requested mutation type – no other kind of change in the same variant.
5. Each variant must genuinely differ from the original (no no-ops) and from every other variant.

## Output format
For EACH variant output exactly one block and nothing else:

##### START {n} - {type_id} #####
{full mutated template}
##### END {n} - {type_id} #####

- {n} = progressive number (1..N).
- {type_id} = the identifier of the mutation type you were asked to apply (e.g. "a", "k").
- The block body is the COMPLETE mutated template.
- No comments, no explanations, no Markdown code fences.
```

Messaggio utente (esempio, tipo k) — la definizione del tipo a–k e il template vengono iniettati

```
Apply ONLY the following mutation type to the template below.

Mutation type id: k
Name: HTML Tag Insertion
Definition: Insert a new wrapper/container tag around an existing element (add a hierarchy level).
Generic example:
Before: <tag attr="value">text</tag>
After:  <wrapper>
  <tag attr="value">text</tag>
</wrapper>

Produce exactly N variants, numbered 1..N, each applying THIS mutation type to a DIFFERENT element you choose. Output ONLY the delimited
blocks, with {type_id} = k.

----- ORIGINAL TEMPLATE -----
{contenuto completo del file .html}
----- END ORIGINAL TEMPLATE -----
```

3. Modifiche reali generate da LLM

Nota: questa tabella e un'aggiunta sperimentale, inserita per osservare quali risultati produce la generazione di *modifiche realistiche e libere* da parte dell'LLM (non vincolate al modello di mutazioni statico). 200 mutazioni.

Locator	Success	Fragility	Obsolescence	Not Compiled
Relative	178/200	4/200	3/200	15/200
Absolute	176/200	6/200	3/200	15/200
Robula	182/200	0/200	3/200	15/200

Locator	Success	Fragility	Obsolescence	Not Compiled
Katalon	181/200	1/200	3/200	15/200
Selenium	175/200	7/200	3/200	15/200
Robula+	170/200	12/200	3/200	15/200

Prompt usato (realistico) — system

```
# Prompt – Independent Realistic Angular Mutation (LLM strategy)

## Role
You are a front-end developer working on an Angular 19 template. You make the kind of
small, plausible changes a real developer or designer makes during everyday UI work and
redesigns. You decide entirely on your own what to change and where – there are NO
instructions telling you which element to touch or which kind of edit to apply.

## Task
You receive the complete source of one Angular template. Produce N variants of it. In each
variant you autonomously introduce ONE realistic change of your own choosing, somewhere in
the template. The change should be the sort of thing a developer would actually commit: it
keeps the page working, but it may reshape how the markup is organised, so test locators may
or may not react.

## Spread and depth (variety, not direction)
You decide everything; this only asks for a varied set, NOT where or how to act:
- Across the N variants, vary BOTH where in the template you act and HOW MUCH you change –
  from a minor touch up to a single, more substantial restructuring of a piece of markup.
- Do NOT restrict the set to the smallest cosmetic edits, and do NOT concentrate every
  variant on the same spot. Cover the template broadly.
- No two variants may make essentially the same edit.

## Application independence (CRITICAL)
Operate ONLY on the markup you are given. Do NOT assume anything about which application
this is, its domain, its features, or its data. Do not invent app-specific names. Use
exclusively the tags, classes and attributes already present in the provided HTML. The same
strategy must work unchanged on any Angular application.

## Validity constraints (correctness rules – NOT instructions on what to change)
1. The result MUST still be valid Angular 19 and compile.
2. Do NOT alter the syntax of control flow or bindings: `@if`, `@for`, `*ngrxLet`, `[prop]`,
  `(event)`, `[(two-way)]`, and `{}` interpolations must stay valid.
3. Do NOT touch attributes starting with `x-test` (they are test ground truth).
4. Keep the feature usable: do not delete or disable the interactive elements a user needs.
5. Exactly ONE self-contained change per variant: it may be small or a single larger
  restructuring, but it must be one coherent edit, not several unrelated ones.
6. Each variant must genuinely differ from the original (no no-ops) and from every other variant.

## Output format
For EACH variant output exactly one block and nothing else:

##### START {n} - {label} #####
{full mutated template}
##### END {n} - {label} #####

- {n} = progressive number (1..N).
- {label} = a short free-text word YOU choose to describe the change you made (your own words).
- The block body is the COMPLETE mutated template.
- No comments, no explanations, no Markdown code fences.
```

Messaggio utente — viene iniettato il template

```
Here is the complete source of one Angular template.
Produce exactly N variants of it, numbered 1..N, each in its own delimited block exactly as specified in your instructions. Decide entirely on
your own what to change in each variant. Output ONLY the blocks, nothing else.

----- ORIGINAL TEMPLATE -----
{contenuto completo del file .html}
----- END ORIGINAL TEMPLATE -----
```